

IPL WINNER PREDICTION APP

Master Product Document

User Features · UI/UX Design · Motion & Interactions · Venue Intelligence · Tech Stack · Build Plan

Author	PAVAN BHARGAV MN
Version	v2.0 — Complete Master Document
Covers	All 14 user features, Dream11 predictor (your idea), H2H predictor (your idea), What-if engine, n8n email automation, motion effects, venue feature engineering, design system, full tech stack, build order
Model	scikit-learn + XGBoost (planned) · season-level classification
Data	2008–2025 real + synthetic · 2025 winner = RCB (corrected)
Target	Antigravity build handoff — ready to implement

01	Project Overview & Problem Statement
02	User-Facing Features — All 14
03	Feature 01 — Season Prediction Dashboard
04	Feature 02 — Dream11 Team Predictor ♦ Your Idea
05	Feature 03 — Head-to-Head Match Predictor ♦ Your Idea
06	Feature 04 — What-If Scenario Engine ♦ Your Idea
07	Feature 05 — Share & Export
08	Design System — Colors, Typography, Principles
09	Motion & Interaction Effects
10	Venue & Environment Intelligence
11	Backend & API Design
12	n8n Automation — Email Report Delivery ♦ Your Idea
13	Tech Stack & Libraries
14	Build Order & Time Estimates

IPL Winner Prediction App

An interactive web application that predicts IPL season winner probabilities using machine learning. Built on a scikit-learn classification model trained on batting stats, bowling stats, match summaries, and points table data spanning 2008–2025. The frontend transforms these predictions into a cinematic, data-rich experience across five major feature modules.

Attribute	Value
Training data	2008–2023 real + 2024–2025 synthetic (RCB won 2025 — corrected)
Teams modelled	10 IPL teams
Model output	Win probability per team, sums to ~1.0
Top prediction	CSK — 42.1% for IPL 2026
New features	Venue-adjusted stats, home win rate, pitch type proxies
Frontend stack	React + Vite + Chart.js + Framer Motion / GSAP
Backend stack	FastAPI + pickle model + Uvicorn
Automation	n8n webhook → Gmail node → user email inbox
Deployment	Render (backend) + Vercel/Netlify (frontend)

Complete feature map

14 user-facing features across 5 modules. Features marked ♦ are directly from your ideas. Everything is designed from the perspective of a user wanting a top-level UI/UX experience.

#	Mod	Tag	Feature	Description	UX
1	Dashboard	CORE	Win probability display	Podium cards for top 3 + full ranked list for all 10 teams. Parallax hero with mouse-track...	[parallax] [count-up]
2	Dashboard	CORE	Team detail panel	Click any team — side panel slides in with big probability number, implied odds, tier badg...	[slide-in] [gauge]
3	Dashboard	CORE	Team probability carousel	Swipeable horizontal carousel of all 10 team cards. Active card snaps to centre and glows ...	[Embla] [swipe]
4	Dream11	YOUR IDEA	Player card picker	Select 11 players from a card grid. Each card shows player name, role, team badge, recent ...	[card flip] [drag select]
5	Dream11	YOUR IDEA	Cards-fly-left prediction trigger	Press Predict → all 11 player cards animate sliding off to the left side. Result panel rev...	[exit animation] [slide-left]
6	Dream11	YOUR IDEA	Full strategy prediction report	After prediction: win probability for your Dream11 team, how they can win, ideal game plan...	[venue-aware] [AI tips]
7	Dream11	YOUR IDEA	Email report via n8n	User enters email → hits 'Send Report' → n8n webhook fires → formatted HTML prediction rep...	[n8n] [webhook]
8	H2H	YOUR IDEA	Team + venue selector	Pick Team A vs Team B from dropdown badges. Select the match venue from all 12 IPL grounds...	[dropdown] [venue picker]
9	H2H	YOUR IDEA	H2H prediction result	Animated split-screen: Team A win % on left, Team B on right. Bar expands from centre outw...	[split animation] [venue H2H]
10	H2H	YOUR IDEA	Venue conditions card	Shows pitch type, avg first innings score, chase win %, spin vs pace wicket balance, groun...	[pitch info] [visual bars]
11	What-If	YOUR IDEA	Live stat sliders	Adjust batting avg, economy rate, NRR, win rate per team via sliders. Model re-predicts li...	[FastAPI] [debounced]

12	What-If	YOUR IDEA	Venue switcher in what-if	Add a venue dropdown to the what-if panel. Switch Chepauk → Wankhede → Eden Gardens and wa...	[venue param] [live shift]
13	Export	CORE	Shareable prediction card	One-click PNG export of a styled prediction card showing top-3 predictions. Ready for Link...	[html2canvas] [PNG]
14	Export	YOUR IDEA	Email full report	After Dream11 prediction — enter email, receive the full analysis report in inbox via n8n ...	[n8n] [formatted HTML]

What the user sees on first load

The dashboard is the home screen. A user lands here and immediately sees which team is most likely to win IPL 2026. No onboarding, no instructions — the data speaks first.

CORE	Win probability display	Podium cards for top 3 + full ranked list for all 10 teams. Parallax hero with mouse-tracking rings. Count-up animation when numbers load.	[parallax] [count-up] [ranked list]
CORE	Team detail panel	Click any team — side panel slides in with big probability number, implied odds, tier badge (hot favourite / contender / long shot), gauge bar, SHAP explanation.	[slide-in] [gauge] [SHAP]
CORE	Team probability carousel	Swipeable horizontal carousel of all 10 team cards. Active card snaps to centre and glows amber. Touch on mobile, arrows on desktop.	[Embla] [swipe] [snap]

Layout structure

Top bar: logo, title, IPL 2026 season badge. Main area: two-column grid. Left column: podium cards (top 3) + full team list table + bar chart. Right column: detail panel (updates on team click). Below: carousel view of all 10 teams as swipeable cards.

Key UX moments on the dashboard

01 Page load

Numbers count up from 0 to their target values with an easeOutQuart curve. Team list rows stagger in with 80ms delay each. The hero CSK ring pulses once on load.

02 Team click

Right panel slides in. Big probability number counts up. Gauge bar fills with team color. Tier badge fades in. All within 400ms total.

03 Carousel swipe

Smooth momentum scroll. Active card scales up 1.05x and gets a colored border glow. Adjacent cards dim to 0.6 opacity. Dots below update.

04 Mouse move on podium

The three podium cards each have different-speed parallax layers. Rings, numbers, and background shift at 3 different speeds creating depth.

Pick your Dream11 — get your win probability + full strategy

The most unique feature in the app. No other cricket prediction tool lets you build your own team as cards, trigger a cinematic prediction animation, and receive a full venue-aware strategy report by email. This is the feature that makes the app genuinely memorable.

YOUR IDEA ♦ YOUR IDEA	Player card picker	Select 11 players from a card grid. Each card shows player name, role, team badge, recent form score. Selected cards highlight and stack at bottom of screen.	[card flip] [drag select] [form badge]
YOUR IDEA ♦ YOUR IDEA	Cards-fly-left prediction trigger	Press Predict → all 11 player cards animate sliding off to the left side. Result panel reveals from the right. Cinematic sports-broadcast transition.	[exit animation] [slide-left] [reveal-right]
YOUR IDEA ♦ YOUR IDEA	Full strategy prediction report	After prediction: win probability for your Dream11 team, how they can win, ideal game plan for the chosen venue, key player matchups, weaknesses to exploit, toss recommendation.	[venue-aware] [AI tips] [strategy]
YOUR IDEA ♦ YOUR IDEA	Email report via n8n	User enters email → hits 'Send Report' → n8n webhook fires → formatted HTML prediction report lands in their inbox within seconds.	[n8n] [webhook] [Gmail node]

The full user flow

01 Choose match

User selects the upcoming match from a dropdown (e.g. CSK vs MI). This loads the available player pool for both squads.

02 Select venue

User picks the venue. This immediately updates the venue conditions card below — pitch type, avg score, chase %, spin balance shown.

03 Build the 11

User clicks player cards to select. Cards flip on select (CSS 3D flip). Counter at bottom shows '7/11 selected'. Captain and vice-captain starred separately.

04 Hit Predict

All 11 cards animate left using CSS transform translateX(-120%). After 400ms, the result panel slides in from the right.

05 Read the report

Result panel shows: win probability (count-up), win condition (batting targets, bowling plan, key matchups), venue strategy, toss recommendation.

06 Email the report

User enters email address. Hits 'Send Report'. Button shows sending spinner, then 'Sent!' confirmation. n8n fires in background — email arrives within 10 seconds.

Player card design

Each player card: 140px × 180px. Team color top strip. Player name (Syne Bold 14px). Role badge (BAT / BOWL / AR / WK). Team code. Recent form score (last 5 matches, color-coded green/amber/red). Selection state: amber border + checkmark overlay. 3D flip animation on select using CSS perspective + rotateY.

What the prediction report includes

Section	Content
Win probability	Your Dream11 team's win % for this match at this venue
Win condition	How your team needs to play to win — batting target, bowling strategy
Venue strategy	Specific advice for the selected ground — bat first or chase, spin vs pace, powerplay plan
Key matchup	One batter vs one bowler highlight — the most important individual contest
Toss recommendation	Bat or bowl first, with reason based on venue dew, pitch, and team composition
Risk factors	What could go wrong — team weaknesses, opponent strengths

Select two teams + venue → get a full match prediction

The model predicts not just season winners but individual match outcomes. The user picks Team A, Team B, and the venue. The model combines team-level stats, head-to-head record at that specific venue, pitch type, and home advantage to generate a win probability split.

YOUR IDEA ♦ YOUR IDEA	Team + venue selector	Pick Team A vs Team B from dropdown badges. Select the match venue from all 12 IPL grounds. Model factors in pitch type, home advantage, historical H2H record.	[dropdown] [venue picker] [12 grounds]
YOUR IDEA ♦ YOUR IDEA	H2H prediction result	Animated split-screen: Team A win % on left, Team B on right. Bar expands from centre outward. Includes H2H record at that specific venue, toss advice, batting/bowling strategy.	[split animation] [venue H2H] [toss tip]
YOUR IDEA ♦ YOUR IDEA	Venue conditions card	Shows pitch type, avg first innings score, chase win %, spin vs pace wicket balance, ground size. Contextualises why the model favours one team.	[pitch info] [visual bars] [static lookup]

UI layout — H2H screen

Two-column team selector at top. Each column has team badge, full name dropdown, recent form. A VS badge in the centre with a subtle glow. Below: venue dropdown with ground icon and city. Big Predict button at the bottom. On predict: animated split-bar result.

The split animation

Result renders as a horizontal bar split in two. Centre starts at 50/50, then animates outward over 1.2 seconds to the actual win split (e.g. CSK 67% | MI 33%). Team colors fill each side. The winning team's side gets a subtle pulse glow. Numbers count up simultaneously.

Venue factors that change the prediction

Venue factor	How it shifts the prediction
Home ground	Home team gets +6–8% win probability boost
Pitch type (spin)	Teams with 3+ quality spinners get boosted at Chepauk, Eden Gardens
Small ground	Batting-heavy teams (high strike rate) boosted at Wankhede, Chinnaswamy
Chase win % > 52%	Teams with strong chasing record get slight boost
Late season (May)	Dew-heavy venues flip advantage to chasing team in evening games

Adjust stats, change venue, watch probabilities shift live

The what-if engine is the most academically impressive feature — it shows that the model has genuinely learned feature relationships, not just memorised outcomes. Any examiner or evaluator who sees this will immediately understand the model's behaviour.

YOUR IDEA ♦ YOUR IDEA	Live stat sliders	Adjust batting avg, economy rate, NRR, win rate per team via sliders. Model re-predicts live via FastAPI. All probability bars update in real time.	[FastAPI] [debounced] [real-time]
YOUR IDEA ♦ YOUR IDEA	Venue switcher in what-if	Add a venue dropdown to the what-if panel. Switch Chepauk → Wankhede → Eden Gardens and watch probabilities shift based on pitch and home advantage.	[venue param] [live shift] [model override]

Sliders available

Slider	Range	Default	Effect on model
Team batting avg	15 – 45	Season avg	Higher avg → higher win prob for batting-strong teams
Economy rate	6.5 – 12.0	Season avg	Lower economy → higher win prob
Net run rate	-1.5 – +1.5	Season NRR	Higher NRR → direct probability boost
Home win rate	20% – 85%	Season rate	Very sensitive — home advantage is top-3 SHAP feature
Venue (dropdown)	12 venues	Neutral venue	Spin venues penalise pace-only attacks; small grounds boost batting

Technical implementation

Sliders are debounced at 300ms. On change: POST to `/api/predict/custom` with feature overrides JSON. FastAPI merges overrides with season defaults, runs `model.predict_proba()`, returns updated dict. React updates all 10 probability bars simultaneously with CSS transition.

Shareable card + email report delivery

CORE	Shareable prediction card	One-click PNG export of a styled prediction card showing top-3 predictions. Ready for LinkedIn posts and GitHub README.	[html2canvas] [PNG] [portfolio]
YOUR IDEA ♦ YOUR IDEA	Email full report	After Dream11 prediction — enter email, receive the full analysis report in inbox via n8n automation. Includes win probability, strategy, venue tips.	[n8n] [formatted HTML] [inbox]

Color palette

Dark-first system. One warm amber accent (#f4a623) anchors all highlights, CTAs, and key numbers. Semantic team colors are assigned once and used consistently everywhere.

	Page background	#0a0c10	Root dark background — every page
	Surface	#0f1117	Cards, panels, odd table rows
	Border default	#1e2130	Card and row borders, dividers
	Amber — primary	#f4a623	Headings, probability numbers, CTAs, rank labels
	Orange — accent only	#e8460b	Top-bar gradient, rank-1 bar fill
	Text primary	#e8e6df	Main body text, team names
	Text muted	#6b6e7a	Labels, section headers, secondary info
	Blue	#378ADD	GT team, info badges, links
	Teal	#1D9E75	LSG team, success states, new feature tags
	Purple	#7F77DD	KKR team, what-if feature color
	Yellow	#f4c430	CSK team color
	Deep blue	#005daa	MI team color
	Red-orange	#e8460b	RCB team color
	Pink	#e91e8c	RR team color
	Orange	#f97316	SRH team color
	Purple	#9b5de5	KKR team color

Typography

Role	Font	Size	Weight	Usage
Page / hero title	Syne	28–32px	800	Cover, hero section probability number
Section heading	Syne	15px	700	Dashboard section labels
Card title	Syne	13–14px	500	Team names, feature titles, player names
Body text	Syne	12–13px	400	Descriptions, report content
Big data numbers	JetBrains Mono	28–52px	800	Win %, odds, count-up displays

Small labels / badges	JetBrains Mono	9–12px	500	Tier badges, axis ticks, stack pills
Code / mono info	JetBrains Mono	10px	400	API endpoints, feature names

Design principles

Dark-first	All surfaces use dark backgrounds. No white panels anywhere. Light mode is out of scope.
Data density	Every pixel earns its place. No decorative padding, no filler content.
Monospace numbers	All probability values, odds, and stats use JetBrains Mono for column alignment.
Semantic color	Color encodes meaning. Each team has exactly one color, used consistently throughout the app.
Progressive disclosure	Summary first, detail on click. No information overload on initial load.
Micro-interactions	Hover: border brightens + 2px translateY lift. Click: panel updates with 0.4s ease.
Mobile-first	Carousel and card layouts work on touch. Parallax and tilt effects disabled on mobile.
Accessibility	ARIA labels on all charts, focus rings on all interactive elements, keyboard navigable.

6 motion effects — what they are and how to implement

01 Count-up probability animation 1 hour

On team select or page load, probability numbers count up from 0 to target using `easeOutQuart` curve. 15 lines of `requestAnimationFrame` code. Highest quality-to-effort ratio of all effects.

[rAF] [easeOutQuart] [pure JS]

02 Scroll reveal on team list rows 2 hours

Each row slides up and fades in with 80ms staggered delay as user scrolls. `IntersectionObserver` + CSS `translateY`. No library needed.

[IntersectionObserver] [CSS] [translateY]

03 Dream11 card fly-left animation 3 hours

Player cards translate -120% on X axis with staggered 50ms delay per card, `cubic-bezier(0.4,0,0.2,1)`. Result panel enters from right simultaneously.

[CSS transform] [stagger] [cubic-bezier]

04 Mouse-track parallax on podium 3 hours

Three layers (rings, numbers, background dots) move at speed multipliers 0.3x / 0.5x / 0.8x relative to mouse position. Creates genuine depth. Disabled below 768px.

[mousemove] [speed layers] [disabled mobile]

05 3D card tilt on podium top-3 30 min

`VanillaTilt.js` on the top-3 podium cards. Cards tilt toward cursor in 3D. 2 lines to implement after install. Max tilt 12deg, perspective 800px.

[VanillaTilt.js] [perspective] [max-tilt]

06 Custom glowing cursor 1 hour

Replace default cursor with 8px amber dot + 28px trailing ring (2px border). Ring lags behind with 100ms transition. Pure CSS/JS. Desktop only.

[custom cursor] [trailing ring] [amber glow]

Key code patterns

Count-up (most important — implement first):

```
function countUp(el, target, duration = 1200) {
  const start = performance.now();
  const tick = (now) => {
    const t = Math.min((now - start) / duration, 1);
    const ease = 1 - Math.pow(1 - t, 4); // easeOutQuart
    el.textContent = (target * ease).toFixed(1) + '%';
    if (t < 1) requestAnimationFrame(tick);
  };
  requestAnimationFrame(tick);
}
```

Parallax layers (different speed per layer):

```
document.addEventListener('mousemove', (e) => {
  const x = (e.clientX / window.innerWidth - 0.5) * 2;
  const y = (e.clientY / window.innerHeight - 0.5) * 2;
  ring1.style.transform = `translate(${x*18}px, ${y*18}px)`;
  ring2.style.transform = `translate(${x*30}px, ${y*30}px)`;
  numEl.style.transform = `translate(${x*8}px, ${y*8}px)`;
});
```

Why the model needs venue features

Your current model aggregates stats across a full season — blind to WHERE those runs were scored. A batting avg of 28 at Wankhede (small flat ground) is less impressive than 28 at Chepauk (slow turner). Venue-adjusted features fix this and are expected to be top-3 SHAP features after retraining.

12 new features added to the model

Feature	Type	Derived from	SHAP rank (expected)
home_win_rate	float 0–1	Match results + venue	Top 3
home_away_delta	float	home_win_rate – away_win_rate	Top 5
venue_adj_batting_avg	float	raw_avg / (venue_avg / 170)	Top 3
venue_adj_economy	float	raw_economy × (venue_avg / 170)	Top 5
chase_win_pct_avg	float 0–1	Venue lookup, weighted by matches	Medium
spin_exposure_avg	float	Avg spin_wkt_pct across team venues	Medium
ground_size_score	1–3	S=1, M=2, L=3, avg across venues	Medium
coastal_match_pct	float 0–1	% matches at coastal venues	Lower
altitude_score	float	Avg altitude_m across venues	Lower
toss_win_rate	float 0–1	Toss results per season	Medium
late_season_match_pct	float 0–1	% matches in May	Medium
late_season_win_rate	float 0–1	Win rate in May matches only	Medium

All 12 IPL grounds — venue metadata

Venue	City	Home	Sz	Coastal	Avg1st	Spin %	Chase %
Wankhede Stadium	Mumbai	MI	S	Yes	182	28	52
MA Chidambaram Stadium	Chennai	CSK	M	Yes	163	55	42
M Chinnaswamy Stadium	Bengaluru	RCB	S	No	185	30	48
Eden Gardens	Kolkata	KKR	L	No	168	38	55
Arun Jaitley Stadium	Delhi	DC	M	No	172	35	50

Rajiv Gandhi Intl Stadium	Hyderabad	SRH	M	No	170	40	53
Sawai Mansingh Stadium	Jaipur	RR	M	No	167	42	51
Punjab Cricket Association	Mohali	PBK S	M	No	171	33	49
Narendra Modi Stadium	Ahmedabad	GT	L	No	174	36	47
BRSABV Ekana Stadium	Lucknow	LSG	M	No	169	37	50
DY Patil Stadium	Navi Mumbai	N/A	M	Yes	176	29	51
Brabourne Stadium	Mumbai	N/A	S	Yes	179	27	50

FastAPI — minimal, stateless, model-serving

The backend is intentionally minimal. Model loads once at startup from a pickle file and stays in memory. All endpoints are stateless. Frontend sends feature overrides; backend merges with season defaults, runs `predict_proba()`, returns JSON.

Endpoint	Method	Purpose	Used by
GET /api/predict	GET	Season win probabilities (default features)	Dashboard load
POST /api/predict/custom	POST	Custom feature overrides → updated probabilities	What-if sliders
POST /api/predict/h2h	POST	Team A vs Team B + venue → match win %	H2H predictor
POST /api/predict/dream11	POST	11 player IDs + venue → team win % + strategy	Dream11 predictor
GET /api/teams	GET	Team metadata, colors, full names	All features
GET /api/venues	GET	12 ground metadata objects	H2H, Dream11, What-if
GET /api/history	GET	Season winner history 2008–2025	Season timeline
GET /api/explain/{team}	GET	SHAP values per feature for a team	Detail panel
POST /api/email/report	POST	Trigger n8n webhook with report payload	Dream11 email
GET /health	GET	Status check	Monitoring

Model serving pattern

```
# main.py — load once at startup
import pickle, numpy as np
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=['*'], allow_methods=['*'])

model = pickle.load(open('ipl_model.pkl', 'rb'))
scaler = pickle.load(open('scaler.pkl', 'rb'))
TEAMS = ['CSK', 'GT', 'MI', 'LSG', 'RCB', 'DC', 'RR', 'KKR', 'SRH', 'PBKS']

@app.post('/api/predict/custom')
async def predict_custom(overrides: dict):
    features = SEASON_DEFAULTS.copy()
    features.update(overrides)
    X = scaler.transform([list(features.values())])
    probs = model.predict_proba(X)[0]
    return dict(zip(TEAMS, [round(p, 6) for p in probs]))
```

How the Dream11 report gets to the user's inbox

You already have n8n set up with Gmail and Twilio nodes from the Technoverse hackathon. The email report flow reuses the same webhook trigger pattern — it's the same architecture, different payload.

n8n workflow — step by step

01 Webhook node (POST /report)

FastAPI calls this after Dream11 prediction. Payload: { email, team_name, win_pct, strategy, venue, player_list, toss_tip }

02 Format HTML node (Function node)

Transforms the JSON payload into a styled HTML email template. Amber colour scheme, team badge, win %, strategy sections.

03 Gmail node

Sends the formatted HTML email to the user's address. Subject: 'Your IPL Dream11 Prediction — [TEAM] vs [OPP] at [VENUE]'.

04 Response node

Returns { success: true, message_id: '...' } back to FastAPI. FastAPI responds to frontend with 200 OK.

FastAPI → n8n call

```
@app.post('/api/email/report')
async def send_report(payload: ReportPayload):
    import httpx
    n8n_url = os.getenv('N8N_WEBHOOK_URL')
    async with httpx.AsyncClient() as client:
        resp = await client.post(n8n_url, json=payload.dict(), timeout=10)
    return {'status': 'sent', 'code': resp.status_code}
```

Email report template sections

Section	Content	Styling
Header	Team badge + 'Your IPL Dream11 Prediction'	Amber top border, dark bg
Match info	Team A vs Team B · Venue · Date	Muted text, mono font
Win probability	Big number + gauge bar	52px Mono, amber color
Win condition	How your team can win this match	Body text, bullet points
Venue strategy	Specific ground tips	Indented card, teal accent

Toss recommendation	Bat or bowl first + reason	Badge: green/red
Player list	Your selected 11 with roles	Grid layout
Footer	Powered by IPL Prediction App · Pavan Bhargav MN	Muted, small

Complete stack — every library and why

Layer	Library	Version	Purpose
Model	scikit-learn	1.3+	Base classification, probability output
Model (upgrade)	XGBoost	2.0+	Better accuracy + SHAP support
Explainability	SHAP	0.44+	Feature attribution, waterfall charts
Backend	FastAPI	0.110+	REST API, model serving, CORS, async
Backend server	Uvicorn	0.27+	ASGI server
HTTP client	httpx	0.27+	Async calls from FastAPI to n8n
Frontend framework	React + Vite	18+ / 5+	Component UI, fast dev server
Routing	React Router	6+	URL state, shareable links
Charting	Chart.js	4.4+	Bar charts, radar, line charts
Carousel	Embla Carousel	8+	Team card carousel, touch-native
Motion (option A)	Framer Motion	11+	React animations, layout transitions
Motion (option B)	GSAP + ScrollTrigger	3+	Scroll reveals, timeline animations
3D tilt	VanillaTilt.js	1.8+	Podium card 3D hover effect
Export	html2canvas	1.4+	PNG prediction card export
Fonts	Syne + JetBrains Mono	—	Google Fonts — display + mono
Automation	n8n	latest	Webhook → Gmail email delivery
Deployment backend	Render	free	FastAPI hosting
Deployment frontend	Vercel	free	React/Vite hosting

Recommended implementation sequence

Ordered by dependency and demo value. Each phase is independently demoable. Total estimate for a focused sprint: ~40–50 hours.

Phase 1: Foundation		4–6 hrs
Export model	pickle.dump(model) + pickle.dump(scaler) — must do first, everything depends on this	
FastAPI skeleton	main.py + /predict + /health + CORS. Test locally with uvicorn.	
React + Vite setup	npm create vite@latest + install Chart.js, Framer Motion, Embla	
Static probability display	Hardcode model output JSON, render team list — no backend needed yet	
Phase 2: Core dashboard		8–10 hrs
Count-up animation	15-line rAF countUp() function — implement on page load	
Team detail panel	Click team → right panel updates. Slide-in CSS animation.	
Bar chart	Chart.js bar chart, dark theme, amber bars, team colors	
Scroll reveal	IntersectionObserver + staggered CSS on team list rows	
Team carousel	Embla Carousel + team cards + dots navigation	
Phase 3: Motion effects		6–8 hrs
Parallax hero	mousemove handler + 3-layer speed multipliers on podium	
3D card tilt	VanillaTilt.js on top-3 podium cards (30 min)	
Custom cursor	8px dot + 28px trailing ring, amber glow, desktop only	
Dream11 card fly	translateX(-120%) stagger on predict button press	
Phase 4: New features		14–16 hrs
Dream11 player picker	Player card grid, select 11, captain/vice badges, flip animation	

Dream11 prediction result	POST /api/predict/dream11 → strategy report render
H2H predictor	Team A + Team B dropdowns + venue + split animation result
What-if sliders	5 sliders + venue dropdown + debounced FastAPI calls
Venue breakdown panel	Home vs away split bars, ground conditions card
Phase 5: Automation + deploy	
	6–8 hrs
n8n email workflow	Webhook → Function (HTML template) → Gmail node
Email button in Dream11	Enter email + send button + success toast
Deploy backend	Push to GitHub → connect Render → set start command
Deploy frontend	Vercel deploy, set VITE_API_URL env var
PNG export card	html2canvas on prediction card → download link

Correction note + generated files

CORRECTION APPLIED: 2025 IPL winner is RCB (Royal Challengers Bengaluru), not MI. RCB's team strength prior updated from 0.60 to 0.72 across all synthetic data generation. All four CSV files and the venue feature pipeline reflect this correction.

IPL winner history 2008–2025 (corrected)

Year	Winner	Titles (cumulative)
2008	RR — Rajasthan Royals	RR: 1
2009	DC — Deccan Chargers	DC: 1
2010–2011	CSK — Chennai Super Kings	CSK: 2
2012, 2014	KKR — Kolkata Knight Riders	KKR: 2
2013, 2015, 2017, 2019, 2020	MI — Mumbai Indians	MI: 5
2016	SRH — Sunrisers Hyderabad	SRH: 1
2018, 2021, 2023	CSK — Chennai Super Kings	CSK: 5 total
2022	GT — Gujarat Titans	GT: 1
2024	KKR — Kolkata Knight Riders	KKR: 3 total
2025	RCB — Royal Challengers Bengaluru ♦ CORRECTED	RCB: 1 (first ever title)

Generated CSV files (alongside this PDF)

File	Rows	Columns	Description
batting_2024_2025_synthetic.csv	220	11	Player batting stats, both seasons, RCB corrected
bowling_2024_2025_synthetic.csv	140	10	Player bowling stats, both seasons
match_summary_2024_2025_synthetic.csv	120	8	Match results with NRR
points_table_2024_2025_synthetic.csv	20	7	Season points table both years
venue_metadata.csv	12	9	Static ground metadata for all IPL venues
match_venue_2024_2025.csv	~480	15	Match data with full venue features

venue_season_features_2024_2025.csv	20	10	Season-level venue-adjusted features per team
-------------------------------------	----	----	---

End of Document

PAVAN BHARGAV MN · IPL Winner Prediction App · Master Product Document v2.0

Prepared for handoff to Antigravity